

INTRODUCCIÓN A LA PROGRAMACIÓN ORIENTADA A OBJETOS

Asociación y Dependencia de Clases

Depto. de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur, Bahía Blanca

Identidad y Equivalencias entre clases asociadas

Termostato

<<Atributos de instancia>>
panel, actual: entero

<<Constructor>>

Termostato(p, a: entero)

<<Comandos>>

establecerPanel(p: entero)

establecerActual(a: entero)

copy(t: Termostato)

<<Consultas>>

obtenerPanel(): entero

obtenerActual(): entero

regulado(): boolean

equals(t: Termostato): boolean

clone(): Termostato

toString(): String

En un sistema de automatización de viviendas se modelan diferentes tipos de dispositivos, uno de los más simples es un termostato.

Identidad y Equivalencias entre clases asociadas

Termostato

<<Atributos de instancia>>
panel, actual: entero

<<Constructor>>

Termostato(p, a: entero)

<<Comandos>>

establecerPanel(p: entero)

establecerActual(a: entero)

copy(t: Termostato)

<<Consultas>>

obtenerPanel(): entero

obtenerActual(): entero

regulado(): boolean

equals(t: Termostato): boolean

clone(): Termostato

toString(): String

establecerPanel(p: entero)

El parámetro lo ingresó el usuario.

establecerActual(a: entero)

El parámetro fue leído de un sensor.

regulado(): boolean

El termostato está regulado si la última temperatura sensada es la que estableció el usuario en el panel.

equals(t: Termostato): boolean

Requiere t ligado.

Identidad y Equivalencias entre clases asociadas

```
class Termostato{
    //Atributos de instancia
    private int panel;
    private int actual;
    //Constructor
    public Termostato(int p,int a){
        panel = p;
        actual = a;
    }
}
```

Identidad y Equivalencias entre clases asociadas

```
//Comandos
public void establecerPanel(int p){
    /*La establece el usuario a través del panel*/
    panel = p;
}
public void establecerActual(int p){
    /*La lee un sensor*/
    actual = p;
}
public void copy (Termostato t){
    if (t != null){
        panel = t.obtenerPanel();
        actual = t.obtenerActual();
    }
}
```

Identidad y Equivalencias entre clases asociadas

```
//Consultas
public int obtenerPanel(){
    return panel;
}
public int obtenerActual(){
    return actual;
}
public boolean regulado(){
    /*El termostato está regulado si la última
    temperatura sensada es la que estableció el
    usuario en el panel*/
    return panel == actual;
}
```

Identidad y Equivalencias entre clases asociadas

```
...
public Tmostato clone(){
    return new Tmostato(panel, actual);
}

public boolean equals(Tmostato t){
    //Requiere t ligado
    return panel == t.obtenerPanel() &&
        actual == t.obtenerActual();
}

public String toString(){
    return panel + " " + actual;
}
}
```

```
0 100
0 011
0 110
0 110
0 100
0 011
0 110
0 110
0 01
1 11
0 0
1
```

Identidad y Equivalencias entre clases asociadas

Termostato
<<Atributos de instancia>> panel, actual: entero
<<Constructor>> Termostato(p, a: entero)
<<Comandos>> establecerPanel(p: entero) establecerActual(a: entero) copy(t: Termostato)
<<Consultas>> obtenerPanel(): entero obtenerActual(): entero regulado(): boolean equals(t: Termostato): boolean clone(): Termostato toString(): String

Termotanque
<<Atributos de instancia>> mechero: boolean capacidad: entero termostato: Termostato
<<Constructor>> Termotanque(t: Termostato, c: entero)
<<Comandos>> encender() apagar() establecerTermostato(t: Termostato) establecerCapacidad(c: entero) copy(m: Termotanque)

Identidad y Equivalencias entre clases asociadas

Termotanque(t: Termostato, c: entero) Requiere t ligado, el mechero se inicializa en falso
establecerTermostato(t: Termostato) Requiere t ligado
copy (m: Termotanque) Requiere m ligado

Termotanque
<<Atributos de instancia>> mechero: boolean capacidad: entero termostato: Termostato
<<Constructor>> Termotanque(t: Termostato, c: entero)
<<Comandos>> encender() apagar() establecerTermostato(t: Termostato) establecerCapacidad(c: entero) copy(m: Termotanque)

Identidad y Equivalencias entre clases asociadas

equals(t: Termotanque): boolean Requiere t ligado
--

Termotanque
<<Atributos de instancia>> mechero: boolean capacidad: entero termostato: Termostato
<<Consultas>> encendido(): boolean obtenerCapacidad(): entero obtenerTermostato(): Termostato equals(t: Termotanque): boolean clone(): Termotanque toString(): String

Identidad y Equivalencias entre clases asociadas

```
class Termotanque{
    //Atributos de instancia
    private boolean mechero;
    private int capacidad;
    private Termostato termostato;
    //Constructor
    public Termotanque (Termostato t , int c){
        //Requiere t ligado
        mechero = false;
        capacidad = c;
        termostato = t;
    }
}
```

```
0 1100
1 0011
0 1110
0 1110
0 1100
1 0011
0 1110
0 1110
1 001
1 11
0 0
1
```

Identidad y Equivalencias entre clases asociadas

```
...
//Comandos
public void encender(){
    mechero = true;
}

public void apagar(){
    mechero = false;
}

public void establecerCapacidad(int c){
    capacidad = c;
}

public void establecerTermostato(Termostato t){
    //Requiere t ligado
    termostato = t;
}
}
```

```
0 1100
1 0011
0 1110
0 1110
0 1100
1 0011
0 1110
0 1110
1 001
1 11
0 0
1
```

Identidad y Equivalencias entre clases asociadas

```
//Consultas
public boolean encendido() {
    return mechero;
}
public int obtenerCapacidad() {
    return capacidad;
}
public Termostato obtenerTermostato() {
    return termostato;
}
```

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```



Identidad y Equivalencias entre clases asociadas

- El comando **copy** modifica el estado interno del objeto que recibe el mensaje con el estado interno del objeto que pasa como parámetro.
- En la **copia superficial** los dos termotanques quedan asociados a un mismo termostato. La referencia al termostato del termotanque que pasa como parámetro, se asigna al atributo de instancia **termostato**, del termotanque que recibe el mensaje.
- En la **copia profundidad** los dos termotanques quedan asociados a distintos termostatos, equivalentes entre sí. El estado interno del termostato del termotanque que pasa como parámetro, se copia en el estado interno del termostato asociado al termotanque que recibe el mensaje.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```



Identidad y Equivalencias entre clases asociadas

```
public void copy(Termotanque t) {
    //Requiere t ligado
    mechero = t.encendido();
    capacidad = t.obtenerCapacidad();
    termostato = t.obtenerTermostato();
}
```

Superficial

```
public void copy(Termotanque t) {
    //Requiere t ligado
    mechero = t.encendido();
    capacidad = t.obtenerCapacidad();
    termostato.copy(t.obtenerTermostato());
}
```

Profundidad

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```



Identidad y Equivalencias entre clases asociadas

La consulta **equals** compara el estado interno del objeto que recibe el mensaje con el estado interno del objeto que pasa como parámetro.

La **igualdad superficial** computa true si los dos termotanques tienen el mismo valor en los atributos mechero y capacidad, y están asociados a un mismo termostato, esto es las referencias son iguales.

La **igualdad en profundidad** computa true si los dos termotanques tienen el mismo valor en los atributos mechero y capacidad, y están asociados a termostatos equivalentes.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```



Identidad y Equivalencias entre clases asociadas

```
public boolean equals (Termotanque t) {
    //Requiere t ligado
    return mechero == t.encendido() &&
           capacidad == t.obtenerCapacidad() &&
           termostato == t.obtenerTermostato();
}
```

Superficial

```
public boolean equals (Termotanque t) {
    //Requiere t ligado
    return (mechero == t.encendido() &&
           capacidad == t.obtenerCapacidad() &&
           termostato.equals(t.obtenerTermostato()));
}
```

Profundidad

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```



Identidad y Equivalencias entre clases asociadas

La consulta **clone** crea y retorna como resultado un objeto con el mismo estado interno que el objeto que recibe el mensaje.

En la **clonación superficial** los dos termotanques quedan asociados a un mismo termostato.

En la **clonación en profundidad** el nuevo termotanque queda asociado a un nuevo termostato, que es un clon del termostato del termotanque que recibe el mensaje. Así, los termostatos de los dos termotanques, tienen distinta identidad pero son equivalentes.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```



Identidad y Equivalencias entre clases asociadas

```
public Termotanque clone() {
    Termotanque t = new Termotanque(termostato,
                                     capacidad);

    if (mechero)
        t.encender();
    return t;
}
```

Superficial

```
public Termotanque clone() {
    Termotanque t = new Termotanque(termostato.clone(),
                                     capacidad);

    if (mechero)
        t.encender();
    return t;
}
```

Profundidad

Identidad y Equivalencias entre clases asociadas

```
public String toString() {
    String estado;
    if (mechero) estado = "encendido";
    else estado = "apagado";
    return estado + " " + capacidad + " " +
        + termostato.toString();
}
```

```
01100
0011
0110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Identidad y Equivalencias entre clases asociadas

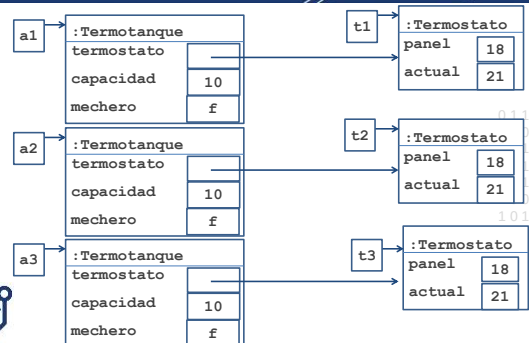
```
Termostato t1 = new Termostato (18,21);
Termostato t2 = new Termostato (18,21);
Termostato t3 = t1.clone();
```

```
TermoTanque a1 = new TermoTanque(t1,10);
TermoTanque a2 = new TermoTanque(t2,10);
TermoTanque a3 = new TermoTanque(t3,10);
```

```
boolean b1 = a1.equals(a2);
boolean b2 = a1.equals(a3);
```

```
01100
0011
0110
01110
01100
1001
1 11
0 0
1
```

Identidad y Equivalencias entre clases asociadas



```
01100
011
10
10
100
011
10110
10
1
1
```

Identidad y Equivalencias entre clases asociadas

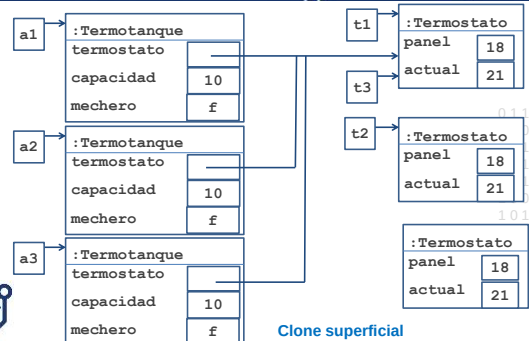
```
Termostato t1 = new Termostato (18,21);
Termostato t2 = new Termostato (18,21);
Termostato t3 = t1;
```

```
TermoTanque a1 = new TermoTanque(t1,10);
TermoTanque a2 = new TermoTanque(t3,10);
TermoTanque a3 = a1.clone();
```

```
boolean b1 = a1.equals(a2);
boolean b2 = a1.equals(a3);
```

```
01100
0011
0110
01110
01100
1001
1 11
0 0
1
```

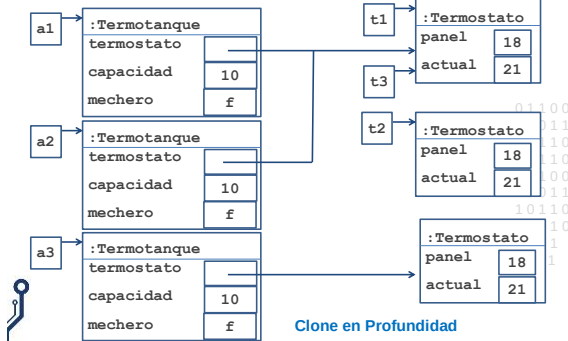
Identidad y Equivalencias entre clases asociadas



```
01100
011
10
10
100
011
10110
10
1
1
```

Clone superficial

Identidad y Equivalencias entre clases asociadas



REPRESENTACIÓN EN MEMORIA

Representación por Referencia

- Cada objeto de software modela a un objeto del problema identificado en la etapa de diseño.
- El estado interno de un objeto puede contener referencias a otros objetos, de modo que un sistema complejo puede modelarse a partir de objetos simples.
- La modificación de la representación de un objeto no afecta a la representación de los objetos que lo referencian.
- En este caso si cambia la representación interna de la clase **Termostato**, la modificación no afecta a la clase **Termotanque**.